CIM — Component Identification Module

CIM — Component Identification Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Dependency Standard (SDS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Component Identification Governance

Module ID: SLGA-SDS-CIM-001

Origin ID: OOF.CM.SLGA.03.01.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Component Identification Module (CIM) governs how software components

and other software-related resources upon which a software system materially relies are discovered, distinguished, classified, identified, and represented as governed dependency objects.

CIM establishes the methodological conditions under which a software

component may be recognized as an identifiable constituent of a software dependency relationship, regardless of whether that component is directly declared, indirectly introduced, dynamically resolved, externally provided, generated, embedded, or accessed through a service interface.

It governs the relationship between the dependent software object, the

discovered component, its distinguishable identity, relevant version or state, functional role, dependency type, observed reliance, and supporting identification evidence.

CIM distinguishes component discovery from component identification,

declared dependency information from actual software reliance, and technical component presence from sufficiently supported recognition of a governed dependency object.

It applies to software libraries, packages, frameworks, plugins, runtime

components, build tools, development resources, software services, interfaces, generated components, embedded software resources, and material direct or transitive dependencies.

Its principal output is:

Governed Software Component Identification State

Its foundational distinction is:

Component Discovery ≠ Governed Component Identification.

Operational Role

The operational role of CIM is:

To ensure that material software components and dependency resources are sufficiently identifiable, distinguishable, classified, and connected to the software systems that rely upon them before their origin, suitability, introduction, change, risk, or continuity can be competently governed.

CIM establishes the component-identification foundation of SDS.

It answers the first dependency-governance question:

What does this software system materially depend on?

CIM does not independently determine whether a component should be used, whether its origin is trustworthy, whether its inclusion was authorized, or whether it is suitable for the intended software context.

It establishes the identifiable dependency objects upon which those

subsequent assessments depend.

Core Question

How can every material software component and dependency resource be identified and distinguished across direct, transitive, generated, embedded, and externally provided dependency relationships without treating package declarations, component names, or technical discovery results as sufficient proof of actual software reliance?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

CIM operates within:

Software Dependency Standard (SDS)

The SDS internal governance flow is:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

CIM governs:

Component Identification

It is the first Core Module of SDS.

Its principal upstream references include:

Governed Software Requirements Baseline

from Software Requirements Standard (SRS).

Governed Software Creation State

from Software Creation Standard (SCS).

Relevant software artifact and composition references may also be received from CRIM and SACM within SCS.

Its principal output is:

Governed Software Component Identification State

This state supports:

DOTM — Dependency Origin & Traceability Module

and the remaining SDS Core Modules.

CIM may be invoked repeatedly as dependency relationships emerge or change.

• Governed Space

Software Component Identification Governance

This space governs the recognition and identification of material software

components and dependency resources.

Its principal dimensions include:

• Dependent software object identification

• Component discovery

• Component recognition

• Component identity

• Component distinguishability

• Component classification

• Dependency relationship identification

• Direct and indirect dependency recognition

• Version and state identification

• Component role identification

• Component boundary identification

• Component occurrence and instance recognition

• Generated and dynamically resolved components

• Identification evidence

• Identification uncertainty

• Identification conflicts

• Identification gaps

• Identification drift

• Identification continuity

• Identification governance states

The governed space is independent of any particular package manager, software repository, programming language, runtime platform, or software development methodology.

Component Identification Governance Object

Every CIM assessment shall establish a Component Identification Governance

Object.

The object represents a software component or dependency resource whose

identity and relationship to a dependent software object are being assessed.

It should identify:

Identification Object Identity: The identifiable governance assessment.

Dependent Software Object: The software system or subsystem under

assessment.

Candidate Component: The discovered or reported dependency resource.

Component Identity: The distinguishable component reference.

Component Type: The relevant component classification.

Component Role: The function performed in relation to the dependent

software object.

Dependency Relationship: The observed or claimed reliance relationship.

Version or State: The relevant technical identity condition.

Occurrence Context: The environment or scope in which the component

appears.

Identification Evidence: Evidence supporting component recognition.

Identification Uncertainty: Material unresolved identification conditions.

Identification Governance State: The current recognized identification

condition.

The identification object shall remain distinguishable from the underlying

software artifact.

Component Identification Object ≠ Component Artifact.

Dependent Software Object Identification

CIM shall identify the software object whose dependency relationships are being examined.

The dependent software object may be:

• An application

• A software service

• A software component

• A software platform

• A development tool

• A build system

• A generated software artifact

• An autonomous software system

• A defined software subsystem

The dependent software object shall be sufficiently identified to establish

the scope of component recognition.

A component may be a dependency of one software object while serving as a dependent software object in another relationship.

Dependency Role ≠ Permanent Component Classification.

Component Discovery

CIM shall support discovery of material software components and dependency

resources.

Discovery sources may include:

• Package manifests

• Lockfiles

• Repository contents

• Build definitions

• Runtime observations

• Software composition records

• Service integration records

• Generated code

• Dependency scanners

• Deployment configurations

• Relevant development evidence

Discovery produces candidate component information.

A discovery result shall not automatically be recognized as a sufficiently

identified component.

Component Discovery ≠ Governed Component Identification.

Component Recognition

CIM shall establish whether a discovered resource qualifies as a material

software component or dependency resource within the assessment scope.

Recognition should consider:

• Whether the resource is identifiable

• Whether a material reliance relationship exists or is claimed

• The relevant software context

• The component's functional or operational role

• The available evidence

• Material uncertainty

A component may be recognized as a candidate while its exact identity remains unresolved.

Candidate Component Recognition ≠ Verified Component Identity.

Component Identity

CIM shall establish sufficiently distinguishable identities for material

components.

Relevant identity information may include:

• Component name

• Package identifier

• Artifact identifier

• Service identifier

• Component version

• Distribution reference

• Repository reference

• Relevant content identifier

• Component namespace

• Component-specific identification evidence

Identity shall be sufficiently precise to distinguish materially different

components or states.

Component Name ≠ Governed Component Identity.

Component Identity Granularity

CIM shall establish the identification granularity appropriate to the

governed dependency relationship.

Granularity may concern:

• Software product

• Package

• Library

• Module

• Plugin

• Service

• Component instance

• Build artifact

• Runtime artifact

• Relevant software version

Insufficient granularity may conceal materially different resources.

Excessive granularity may create identification records without corresponding governance value.

Identification Detail ≠ Identification Adequacy.

Component Classification

CIM shall classify components according to their relevant software

dependency roles.

Classification may include:

Library: Reusable software functionality incorporated or invoked by other

software.

Framework: Software structure or services supporting application

development or execution.

Package: Distributed software unit containing one or more components.

Plugin: Extensible software component integrated through a defined mechanism.

Runtime Component: Software resource required during execution.

Build Component: Software resource required during compilation,

generation, or packaging.

Development Component: Software resource used during software creation.

Service Dependency: External or internal service upon which the software

relies.

Interface Dependency: Software interface whose behavior is materially relied upon.

Platform Dependency: Software platform or environment supporting the

dependent software.

Generated Component: Software component produced by a generation

mechanism.

Embedded Component: Software component incorporated within another software artifact.

Classification shall reflect the component's actual role within the assessed context.

Component Type ≠ Component Governance State.

Component Role Identification

CIM shall identify the role performed by a material component.

Roles may include:

• Functional execution

• Data processing

• Communication

• Authentication support

• User interface

• Storage access

• Compilation

• Testing

• Generation

• Runtime support

• Integration

• Monitoring

A component may perform multiple roles.

The same component may serve different roles across different software systems.

Component Identity ≠ Component Role.

Direct Dependency Identification

CIM shall identify material components directly relied upon by the

dependent software object.

Direct reliance may arise through:

• Explicit imports

• Declared packages

• Linked libraries

• Service integrations

• Runtime requirements

• Direct tool invocation

• Explicit platform reliance

A direct dependency shall remain distinguishable from its transitive

dependencies.

Direct Dependency Declaration ≠ Complete Direct Dependency Identification.

Transitive Dependency Identification

CIM shall identify material components introduced indirectly through other dependencies.

A relevant relationship may be:

Dependent Software → Direct Component → Transitive Component

Identification shall preserve the intermediate relationship where sufficiently supported.

A component shall not be excluded solely because it was introduced

indirectly.

Indirect Introduction ≠ Absence of Material Reliance.

Nested Dependency Identification

CIM shall support identification of dependency relationships extending

across multiple levels.

A relevant chain may be:

Application → Framework → Library → Runtime Component → Supporting Service

Identification should preserve material relationship depth.

Where the complete dependency chain cannot be established, the unresolved portion shall remain explicit.

Known Dependency Root ≠ Complete Dependency Chain.

Dynamic Dependency Identification

Some components are selected or resolved dynamically.

Examples may include:

• Runtime-loaded plugins

• Dynamically linked libraries

• Environment-dependent packages

• Remote service discovery

• Conditional component loading

• Automated dependency resolution

CIM shall identify material dynamically resolved components where evidence permits.

A static dependency record shall not automatically be treated as a complete

representation of dynamic software reliance.

Static Dependency Manifest ≠ Actual Runtime Dependency Set.

Generated Component Identification

CIM shall identify material software components introduced through generation mechanisms.

These may include:

• AI-generated libraries

• Generated code modules

• Automated client libraries

• Generated interface implementations

• Code generator outputs

• Autonomous development artifacts

Identification shall preserve the component's distinguishable identity and

relevant generation references.

Detailed generation provenance remains within competent artifact integrity and traceability governance.

Generated Component Existence ≠ Sufficiently Identified Component.

Embedded Component Identification

CIM shall identify material components embedded within larger software artifacts.

Embedded components may not appear as separately declared packages.

Examples may include:

• Bundled libraries

• Vendored source code

• Embedded runtime modules

• Copied software components

• Statically linked components

CIM shall distinguish the embedded component from the composite artifact

where the distinction is material.

Composite Artifact Identity ≠ Complete Constituent Component

Identification.

External Service Dependency Identification

CIM shall identify material software services upon which a software system

relies.

Relevant information may include:

• Service identity

• Interface reference

• Service role

• Dependent software context

• Relevant endpoint or service identifier

• Evidence of actual reliance

A service dependency may exist without any corresponding package installation.

No Installed Package ≠ No Software Dependency.

Platform Dependency Identification

CIM shall identify material software platform dependencies.

These may include:

• Operating systems

• Runtime platforms

• Application servers

• Database platforms

• Container runtimes

• Software execution environments

• Managed software platforms

Identification shall preserve the relationship between the dependent

software and the relevant platform.

Application Portability Claim ≠ Absence of Platform Dependency.

Development and Build Dependency Identification

CIM shall identify material development and build resources where they affect software creation or resulting artifacts.

These may include:

• Compilers

• Build tools

• Code generators

• Testing frameworks

• Package resolution tools

• Development services

• Automated build environments

A component may be absent from the deployed application while remaining material to its creation.

Runtime Absence ≠ Development Dependency Irrelevance.

Component Version Identification

CIM shall identify relevant component versions or states.

Version information may include:

• Declared version

• Resolved version

• Installed version

• Executed version

• Build version

• Distribution version

• Relevant content reference

Where these differ, CIM shall preserve the distinction.

Declared Version ≠ Actual Component Version.

Component Instance Identification

A software system may use multiple instances of the same component.

CIM shall distinguish component identity from component occurrence where material.

Relevant occurrence contexts may include:

• Development environment

• Build environment

• Testing environment

• Production environment

• Multiple service instances

• Separate application deployments

Component Identity ≠ Component Instance Identity.

Component Boundary Identification

CIM shall establish the boundaries of material component identities.

Relevant questions include:

• What constitutes the component?

• Which artifacts belong to it?

• Which elements are separate dependencies?

• Is the component embedded or independently distributed?

• Does the identified object represent a package, service, or composite

artifact?

Boundary uncertainty shall remain explicit.

Package Boundary ≠ Necessarily Component Boundary.

Component Relationship Identification

CIM shall identify material relationships between components.

Relationships may include:

• Direct reliance

• Indirect reliance

• Runtime invocation

• Build participation

• Service consumption

• Component embedding

• Shared platform reliance

• Conditional use

• Alternative use

Identification shall distinguish observed relationships from claimed or

inferred relationships.

Recorded Component Association ≠ Established Dependency Relationship.

Component Identification Across Environments

CIM shall recognize that dependency identities and occurrences may differ across environments.

Relevant environments include:

• Development

• Testing

• Build

• Staging

• Production

• Maintenance

A component discovered in one environment shall not automatically be

assumed to exist in every other environment.

Development Dependency Set ≠ Production Dependency Set.

Component Identification Evidence

CIM shall preserve evidence supporting material identification claims.

Evidence may include:

• Package manifests

• Lockfiles

• Build records

• Runtime observations

• Service integration records

• Repository evidence

• Artifact identifiers

• Component inventories

• Composition records

• Tool-generated reports

Evidence requirements shall reflect component materiality.

Component Inventory Entry ≠ Verified Component Identification.

Identification Evidence Competence

CIM shall assess whether evidence is competent to support the claimed component identity or relationship.

For example:

A package manifest may support a declared dependency.

A lockfile may support a resolved dependency version.

A runtime observation may support actual component execution.

A build record may support component participation in software creation.

These records establish different facts.

Evidence Availability ≠ Evidence Competence.

Identification Evidence Integrity

CIM shall preserve the integrity of material identification evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Version correspondence

• Preservation history

• Material contradictions

• Evidence completeness

Evidence integrity does not independently establish that the component identification claim is correct.

Evidence Integrity ≠ Identification Validity.

Declared and Observed Components

CIM shall distinguish declared components from components actually observed

in relevant software contexts.

Possible conditions include:

• Declared and observed

• Declared but not observed

• Observed but not declared

• Conditionally observed

• Historically observed

• Identification unresolved

Differences may arise through dynamic loading, unused declarations, build conditions, or incomplete discovery.

Declared Component Set ≠ Observed Component Set.

Component Identification Completeness

CIM shall assess whether material components have been sufficiently

identified within the defined scope.

Completeness may concern:

• Direct dependencies

• Transitive dependencies

• Runtime components

• Build dependencies

• Generated components

• Embedded components

• External services

• Platform dependencies

Completeness shall be evaluated relative to the governed software context.

High Discovery Coverage ≠ Complete Component Identification.

Component Identification Consistency

CIM shall assess whether component identification records are mutually consistent.

Potential conflicts include:

• Contradictory versions

• Inconsistent component names

• Different artifact identities

• Conflicting dependency roles

• Incompatible environment records

• Contradictory component boundaries

Consistency does not independently establish historical accuracy or completeness.

Consistent Component Records ≠ Verified Component Reality.

Component Identification Uncertainty

Identification uncertainty may arise where:

• Component identity is incomplete

• Version information is missing

• Component boundaries are unclear

• Runtime reliance cannot be established

• A service identity is ambiguous

• Generated components lack sufficient identifiers

• Transitive relationships remain unknown

CIM shall preserve material uncertainty.

Uncertain Component Identity ≠ Verified Component Identity.

Component Identification Conflict

An identification conflict exists where material evidence supports incompatible component identities or relationships.

Examples may include:

• Different versions reported for the same execution context

• Contradictory component identifiers

• Conflicting runtime and manifest records

• Incompatible component boundaries

• Disagreement between build evidence and delivered artifacts

CIM shall preserve the conflict and its supporting evidence.

Identification Conflict ≠ Permission to Select an Unsupported Identity.

Component Identification Gap

A Component Identification Gap exists where a material software component or dependency relationship cannot be sufficiently identified.

Examples include:

• An unknown runtime library

• An unrecognized external service

• A missing transitive component

• An embedded library with uncertain identity

• A generated component without a distinguishable version

• An unidentified build dependency

CIM shall identify the gap and its material significance.

Missing Component Identification ≠ Proven Component Defect.

Critical Component Identification Gap

A Critical Component Identification Gap exists where insufficient identification prevents competent assessment of a materially consequential dependency.

Examples may include:

• An unidentified component supporting a critical software function

• An unknown library embedded in safety-relevant software

• A critical external service whose actual identity cannot be

• An unrecognized transitive component essential to system operation

established.

CIM shall preserve the critical gap for competent downstream assessment.

It does not independently determine operational suspension, security failure, or liability.

Critical Identification Gap ≠ Automatic Software Rejection.

Component Identification Drift

Component Identification Drift occurs when the actual component set or identity relationships materially diverge from the recognized identification state.

Possible causes include:

• Automatic updates

• Dynamic dependency resolution

• Unrecorded component replacement

• Runtime environment changes

• Generated code changes

• Service substitutions

• Build configuration changes

CIM shall identify material drift and support reassessment.

Previously Identified Component Set ≠ Permanently Identified Component

Set.

Component Identification Change

CIM shall preserve material changes to component identification records.

Changes may include:

• Newly discovered components

• Corrected identities

• Version changes

• Revised component boundaries

• Changed dependency roles

• Removed components

• New evidence

• Resolved conflicts

Historical identification states shall remain reconstructable where material.

Identification Correction ≠ Historical Identification Erasure.

Component Identification Continuity

CIM shall preserve material component identity relationships across

software lifecycle changes.

Continuity may involve:

• Software version changes

• Repository migrations

• Package source changes

• Development environment changes

• Runtime environment changes

• Component replacement

• Supplier changes

• Software modernization

A persistent component name does not independently establish identity continuity.

Name Continuity ≠ Component Identity Continuity.

Component Identification Governance States

CIM shall support identifiable governance states.

Possible states include:

Discovered: A candidate component has been detected.

Under Assessment: Identification conditions are being evaluated.

Identified: The component has a sufficiently distinguishable identity.

Conditionally Identified: Identification depends on explicit conditions.

Partially Identified: Some material identification dimensions remain

unresolved.

Evidence-Limited: Available identification evidence is insufficient.

Version-Unresolved: The relevant component version cannot be established.

Boundary-Unresolved: Material component boundaries remain uncertain.

Relationship-Unresolved: The component's dependency relationship cannot

be sufficiently established.

Conflicted: Material identification evidence contradicts itself.

Identification-Failed: Applicable identification conditions have been

determined not to hold.

Superseded: A later identification state has replaced the earlier state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not be interpreted as dependency acceptance or software

product certification.

Component Identified ≠ Dependency Accepted.

Component Identification Recognition

CIM shall determine whether a material component has been sufficiently

identified for the intended governance purpose.

Recognition should consider:

• Component identity

• Classification

• Role

• Version

• Boundary

• Dependent software context

• Relationship evidence

• Occurrence context

• Evidence competence

• Evidence integrity

• Uncertainty

• Conflicts

• Continuity

Recognition shall remain limited to the component and relationship

conditions actually assessed.

Recognized Component Identity ≠ Complete Dependency Governance.

Component Identification and Origin

CIM identifies components and preserves relevant origin references.

It does not independently establish complete origin provenance.

Detailed origin and traceability governance remains within:

DOTM — Dependency Origin & Traceability Module

A component may be sufficiently identified while its origin remains unresolved.

Component Identity ≠ Verified Component Origin.

Component Identification and Suitability

CIM identifies the components upon which a software system relies.

It does not independently determine whether those components are suitable.

Detailed suitability governance remains within:

DSM — Dependency Suitability Module

A component may be correctly identified while being incompatible, unnecessary, or unsuitable.

Component Identification ≠ Dependency Suitability.

Component Identification and Introduction Authority

CIM may preserve references to the actors or mechanisms that introduced a component.

It does not independently determine whether the introduction was authorized.

Detailed introduction and change governance remains within:

CICM — Component Introduction & Change Module

Component Presence ≠ Authorized Component Introduction.

Component Identification and Continuity

CIM identifies components and relevant dependency relationships.

It does not independently establish whether those components are maintained, replaceable, available, or operationally sustainable.

Detailed continuity and risk governance remains within:

DCRM — Dependency Continuity & Risk Module

Component Identification ≠ Dependency Continuity Assurance.

Component Identification and Operational Reality

CIM shall distinguish declared dependency information from the components

actually participating in relevant software operation.

A software system may:

• Load undeclared runtime components

• Invoke external services

• Use dynamically resolved libraries

• Execute embedded code

• Rely on platform components absent from application manifests

Where actual component participation cannot be established, the limitation shall remain explicit.

Declared Dependency Inventory ≠ Complete Operational Dependency Reality.

Competent operational reality assessment remains within ORA™.

Component Identification Reassessment

CIM shall support reassessment when material identification conditions change.

Triggers may include:

• New dependency discovery

• Version changes

• Software modifications

• Build changes

• Runtime changes

• Generated component changes

• New service integrations

• Evidence conflicts

• Component identification drift

• Dependency relationship changes

Reassessment shall preserve material historical identification information.

Governed Software Component Identification State

The principal output of CIM is:

Governed Software Component Identification State

The state shall preserve sufficient information to identify:

Identification Governance Object: The component or relationship under

assessment.

Dependent Software Object: The software system relying on the component.

Component Identity: The distinguishable component reference.

Component Classification: The relevant component type.

Component Role: The function performed in the dependent software context.

Dependency Relationship: The material reliance relationship.

Component Version: The relevant version or state.

Component Boundary: The identified component scope.

Component Occurrence: The relevant environment or instance.

Direct or Transitive Position: The component's relationship within the dependency structure.

Discovery Source: The source of candidate component information.

Identification Evidence: Evidence supporting the identification.

Evidence Integrity: The condition of material supporting evidence.

Identification Completeness: The sufficiency of identification coverage.

Identification Consistency: The compatibility of material identification

records.

Identification Uncertainty: Material unresolved identification conditions.

Identification Conflicts: Contradictory component identity evidence.

Identification Gaps: Missing material component identification.

Critical Identification Gaps: Material identification failures requiring

competent attention.

Identification Drift: Divergence from the recognized component state.

Identification Governance State: The current recognized identification

condition.

Continuity: Preservation of component identity relationships across

change.

Downstream References: Information required by competent SDS governance

modules.

The state shall not independently establish dependency suitability,

introduction authority, security, quality, or continuity.

Minimum Implementation Framework

A conforming implementation of CIM must provide the following six functional capabilities.

• Dependent Software Object and Component Discovery Governance

Capability to identify the dependent software object, establish discovery

scope, and discover candidate software components across direct, transitive, generated, embedded, runtime, development, build, service, and platform relationships.

The implementation must distinguish discovered candidates from sufficiently identified components.

• Component Identity, Classification, and Boundary Governance

Capability to establish distinguishable component identities, versions,

classifications, roles, and material component boundaries.

The implementation must not treat component names or package labels as sufficient identity evidence where material ambiguity remains.

• Dependency Relationship, Occurrence, and Environment Governance

Capability to identify material reliance relationships, distinguish direct

and indirect dependencies, and preserve relevant component occurrences across development, build, testing, runtime, and deployment environments.

The implementation must distinguish declared relationships from observed relationships.

• Component Identification Evidence and Recognition Governance

Capability to associate identification claims with competent evidence,

evaluate evidence integrity, and determine whether material component identities and relationships are sufficiently supported.

The implementation must preserve unsupported claims and identification uncertainty.

• Identification Gap, Conflict, Drift, and Continuity Governance

Capability to identify material gaps, critical gaps, conflicting component

records, identification drift, and identity continuity across software changes.

The implementation must preserve material historical identification states and corrections.

• Governed Component Identification State and SDS Handoff Governance

Capability to produce a Governed Software Component Identification State

with sufficient identity, relationship, version, classification, evidence, uncertainty, and continuity information for competent downstream SDS governance.

The implementation must not convert component identification into dependency acceptance.

GIE™ Compatibility

CIM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Component discovery

• Dependency candidate identification

• Component identity normalization

• Component classification

• Component role identification

• Direct dependency recognition

• Transitive dependency discovery

• Nested dependency mapping

• Dynamic dependency identification

• Generated component recognition

• Embedded component recognition

• Service dependency identification

• Platform dependency identification

• Build and development dependency identification

• Version correlation

• Component instance identification

• Component boundary analysis

• Cross-environment component comparison

• Identification evidence association

• Identification completeness assessment

• Identification consistency assessment

• Identification gap detection

• Critical identification gap detection

• Identification conflict analysis

• Identification drift detection

• Component identity continuity assessment

• Governed identification state representation

GIE™ may propose candidate identities or infer possible dependency relationships.

Such outputs shall remain distinguishable from sufficiently evidenced

component identities and relationships.

GIE™ must not invent component versions, fabricate dependency relationships, or convert incomplete discovery into verified component identification.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical CIM governance logic is:

Receive Governed Software Requirements Baseline

↓

Receive Governed Software Creation State

↓

Identify Component Identification Governance Object

↓

Identify Dependent Software Object

↓

Establish Component Discovery Scope

↓

Discover Candidate Components and Dependency Resources

↓

Identify Direct and Transitive Dependency Candidates

↓

Identify Generated, Embedded, Dynamic, Service, and Platform Components

↓

Establish Component Identity

↓

Identify Component Classification and Role

↓

Establish Component Version or State

↓

Establish Component Boundary

↓

Identify Component Occurrence and Environment

↓

Identify Material Dependency Relationships

↓

Associate Component Identification Evidence

↓

Assess Evidence Competence and Integrity

↓

Assess Identification Completeness

↓

Assess Identification Consistency

↓

Identify Identification Gaps and Critical Gaps

↓

Identify Identification Conflicts and Uncertainty

↓

Identify Component Identification Drift

↓

Determine Component Identification Governance State

↓

Establish Governed Software Component Identification State

↓

Preserve Component Identification Continuity

↓

Handoff to DOTM and Competent SDS Governance

↓

Reassess When Material Component Conditions Change

The principal output is:

Governed Software Component Identification State

Governance Outputs

CIM may produce:

Component Identification Governance Object

Dependent Software Object Reference

Component Discovery Scope Record

Candidate Component Inventory

Component Recognition Record

Component Identity Record

Component Identity Granularity Assessment

Component Classification Record

Component Role Record

Direct Dependency Identification Record

Transitive Dependency Identification Record

Nested Dependency Map

Dynamic Dependency Identification Record

Generated Component Identification Record

Embedded Component Identification Record

External Service Dependency Identification Record

Platform Dependency Identification Record

Development and Build Dependency Identification Record

Component Version Identification Record

Component Instance Identification Record

Component Boundary Record

Component Relationship Record

Cross-Environment Component Identification Record

Component Identification Evidence Record

Identification Evidence Competence Assessment

Identification Evidence Integrity Assessment

Declared and Observed Component Comparison

Component Identification Completeness Assessment

Component Identification Consistency Assessment

Component Identification Uncertainty State

Component Identification Conflict Record

Component Identification Gap

Critical Component Identification Gap

Component Identification Drift Record

Component Identification Change Record

Component Identification Continuity Record

Component Identification Governance State

Component Identification Recognition Record

Component Origin Reference

Component Suitability Reference

Component Introduction Authority Reference

Component Continuity Reference

Operational Reality Identification Reference

Component Identification Reassessment Record

SDS Identification Handoff Record

The principal output is:

Governed Software Component Identification State

Critical Distinctions

CIM preserves the following distinctions:

Component Discovery ≠ Governed Component Identification

Component Identification Object ≠ Component Artifact

Dependency Role ≠ Permanent Component Classification

Candidate Component Recognition ≠ Verified Component Identity

Component Name ≠ Governed Component Identity

Identification Detail ≠ Identification Adequacy

Component Type ≠ Component Governance State

Component Identity ≠ Component Role

Direct Dependency Declaration ≠ Complete Direct Dependency Identification

Indirect Introduction ≠ Absence of Material Reliance

Known Dependency Root ≠ Complete Dependency Chain

Static Dependency Manifest ≠ Actual Runtime Dependency Set

Generated Component Existence ≠ Sufficiently Identified Component

Composite Artifact Identity ≠ Complete Constituent Component

Identification

No Installed Package ≠ No Software Dependency

Application Portability Claim ≠ Absence of Platform Dependency

Runtime Absence ≠ Development Dependency Irrelevance

Declared Version ≠ Actual Component Version

Component Identity ≠ Component Instance Identity

Package Boundary ≠ Necessarily Component Boundary

Recorded Component Association ≠ Established Dependency Relationship

Development Dependency Set ≠ Production Dependency Set

Component Inventory Entry ≠ Verified Component Identification

Evidence Availability ≠ Evidence Competence

Evidence Integrity ≠ Identification Validity

Declared Component Set ≠ Observed Component Set

High Discovery Coverage ≠ Complete Component Identification

Consistent Component Records ≠ Verified Component Reality

Uncertain Component Identity ≠ Verified Component Identity

Identification Conflict ≠ Permission to Select an Unsupported Identity

Missing Component Identification ≠ Proven Component Defect

Critical Identification Gap ≠ Automatic Software Rejection

Previously Identified Component Set ≠ Permanently Identified Component Set

Identification Correction ≠ Historical Identification Erasure

Name Continuity ≠ Component Identity Continuity

Component Identified ≠ Dependency Accepted

Recognized Component Identity ≠ Complete Dependency Governance

Component Identity ≠ Verified Component Origin

Component Identification ≠ Dependency Suitability

Component Presence ≠ Authorized Component Introduction

Component Identification ≠ Dependency Continuity Assurance

Declared Dependency Inventory ≠ Complete Operational Dependency Reality

Cross-Architecture Boundary

CIM governs software component identification within SDS and SLGA™.

It does not independently establish dependency origin validity, suitability, introduction authority, security, quality, or continuity.

Within SDS:

CIM — Component Identification Module identifies material software

components and dependency resources.

DOTM — Dependency Origin & Traceability Module governs origin,

provenance, and dependency traceability.

DSM — Dependency Suitability Module governs necessity, compatibility, and

suitability.

CICM — Component Introduction & Change Module governs introduction

authority, inclusion, modification, replacement, and removal.

DCRM — Dependency Continuity & Risk Module governs criticality, risks,

maintenance, availability, substitutability, and continuity.

Within SLGA™:

Software Requirements Standard (SRS) governs requirements.

Software Creation Standard (SCS) governs software creation and artifact

formation.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release.

Software Deployment Standard (SDPS) governs deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

OBIDENITY® — Origin-Bound Identity Governance Architecture: Relevant

origin and identity governance.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

GOA™ — Governance Architecture: General governance authority.

CIM preserves component identification relationships without replacing the

competent governance responsibilities of these domains.

Canonical Boundary

CIM begins when a software system contains, uses, invokes, embeds, generates, or otherwise materially relies upon a software component or software-related resource requiring identification.

CIM governs:

• Component identification governance objects

• Dependent software object identification

• Component discovery

• Component recognition

• Component identity

• Identification granularity

• Component classification

• Component roles

• Direct dependencies

• Transitive dependencies

• Nested dependency relationships

• Dynamic dependencies

• Generated components

• Embedded components

• External service dependencies

• Platform dependencies

• Development and build dependencies

• Component versions

• Component instances

• Component boundaries

• Component relationships

• Cross-environment identification

• Identification evidence

• Evidence competence

• Evidence integrity

• Declared and observed components

• Identification completeness

• Identification consistency

• Identification uncertainty

• Identification conflicts

• Identification gaps

• Critical identification gaps

• Identification drift

• Identification changes

• Identification continuity

• Identification governance states

• Identification recognition

• Component origin references

• Component suitability references

• Component introduction authority references

• Component continuity references

• Operational reality identification references

• Identification reassessment

CIM does not independently determine:

• Complete dependency provenance

• Dependency necessity

• Dependency suitability

• Dependency introduction authority

• Dependency acceptance

• Software component security

• Software functional correctness

• Dependency continuity

• Software release readiness

• Deployment authorization

• Legal liability

CIM ends when the relevant component or dependency resource has been represented as a:

Governed Software Component Identification State

with sufficient identity, classification, version, role, relationship,

evidence, uncertainty, and continuity information for competent downstream SDS governance.

Use Case 1 — An AI-Generated Application Contains an Undeclared

External Service Dependency

An organization uses an AI coding assistant to generate a business

application.

The generated application contains a package manifest listing its software

libraries.

The development team reviews the manifest and concludes that the application's dependencies have been identified.

The application compiles successfully and passes its initial tests.

However, part of the generated code makes runtime requests to an external

service.

That service is not represented in the package manifest.

The development team has not recognized it as a material dependency.

The application may fail or behave differently if the external service becomes unavailable or changes its interface.

CIM evaluates:

Dependent Software Object → Discovered Components → Runtime Service

Relationships → Component Identity → Identification Evidence

The assessment identifies:

• The package manifest does not represent the complete dependency set

• A material service dependency exists outside the declared package

• The service must be sufficiently identified before its origin,

inventory. suitability, authority, and continuity can be competently assessed.

The relevant component identification state may be:

Partially Identified

or:

Relationship-Unresolved

until sufficient evidence is established.

The critical distinctions are:

Package Manifest Entry ≠ Complete Dependency Recognition.

No Installed Package ≠ No Software Dependency.

Declared Dependency Inventory ≠ Complete Operational Dependency Reality.

The result: CIM reveals that software dependency identification must reflect actual material reliance, not merely the contents of conventional package declarations.

Use Case 2 — A Critical Software System Contains an Unidentified

Embedded Library

An organization acquires a software platform from an external supplier.

The supplier provides the application, documentation, and a list of

declared third-party packages.

The application operates successfully.

During a later technical assessment, the organization discovers that a material software library was embedded directly into the application's executable package.

The embedded library does not appear as a separate dependency in the supplied package inventory.

Its exact version is unknown.

Its relationship to the delivered software artifact has not been

sufficiently established.

The organization cannot competently assess the component's origin,

compatibility history, maintenance conditions, or relevant risks until the component is identified.

CIM evaluates:

Delivered Software Artifact → Embedded Component → Component Boundary →

Component Identity → Version → Identification Evidence

The assessment identifies:

• The embedded library is a material constituent component

• Its absence from the declared dependency inventory does not eliminate

• The available evidence is insufficient to establish the precise

• Downstream dependency governance requires an explicit identification

the dependency relationship. component version. state.

The relevant component identification state may be:

Version-Unresolved

or:

Evidence-Limited

until sufficient evidence is established.

The critical distinctions are:

Composite Artifact Identity ≠ Complete Constituent Component

Identification.

Component Name ≠ Governed Component Identity.

Component Identified ≠ Dependency Accepted.

The result: CIM prevents an apparently complete software inventory from

concealing unidentified components embedded within delivered software artifacts.

Canonical Closing Statement

Modern software systems are not composed only of the components developers explicitly declare.

They may contain embedded libraries, dynamically resolved packages, generated modules, external services, runtime platforms, build dependencies, and transitive resources extending across multiple software environments.

Some components are visible in package manifests.

Others appear only during execution.

Some are introduced deliberately.

Others emerge through automated generation, integration, or dependency resolution.

A software system may function correctly while its complete material component structure remains insufficiently identified.

Component Discovery ≠ Governed Component Identification.

Component Name ≠ Governed Component Identity.

Declared Dependency Inventory ≠ Complete Operational Dependency Reality.

Composite Artifact Identity ≠ Complete Constituent Component

Identification.

Component Identified ≠ Dependency Accepted.

CIM establishes the first methodological requirement of software dependency

governance:

A software dependency cannot be competently governed while the component

or resource upon which the system relies remains insufficiently identifiable.

This requirement applies regardless of whether the component was selected

by a human developer, introduced by an autonomous agent, embedded by a supplier, or resolved dynamically during execution.

CIM does not assume that every discovered component is legitimate, suitable, secure, or sustainable.

It establishes the identifiable objects and relationships required for

those determinations to become possible.

CIM transforms software component discovery into governed dependency

identification — ensuring that software systems cannot treat incomplete inventories, package declarations, or technical detection results as sufficient substitutes for knowing which material components they actually rely upon.